

Prepared by	Claude Corde
Approved by	To be filled
Status	Draft for Review

SOFTWARE REQUIREMENTS SPECIFICATION

Full lifecycle platform for research project management,
literature,
experiments, collaboration, analysis & reporting

Table of Contents

1. [Introduction](#) - 1.1 Purpose - 1.2 Scope - 1.3 Definitions, Acronyms, and Abbreviations - 1.4 References - 1.5 Overview
2. [Overall Description](#) - 2.1 Product Perspective - 2.2 Product Functions - 2.3 User Classes and Characteristics - 2.4 Operating Environment - 2.5 Design and Implementation Constraints - 2.6 User Documentation - 2.7 Assumptions and Dependencies
3. [Specific Requirements](#) - 3.1 External Interface Requirements - 3.2 Functional Requirements - 3.3 Non-functional Requirements - 3.4 Database Requirements - 3.5 Other Requirements
4. [System Features](#) - 4.1 Project Management - 4.2 Literature Management - 3.3 Experiment Tracking - 4.4 Collaboration Tools - 4.5 Data Analysis and Visualization - 4.6 Report Generation
5. [Other Non-functional Requirements](#) - 5.1 Performance Requirements - 5.2 Safety Requirements - 5.3 Security Requirements - 5.4 Software Quality Attributes
6. [Other Requirements](#)
7. [Appendices](#) - Appendix A: Glossary - Appendix B: Analysis Models
 - B.1 Data Flow Diagrams (DFD)
 - B.2 Entity-Relationship Diagrams (ERD)
 - B.3 Use Case Diagrams
 - B.4 Activity Diagrams
 - Appendix C: To Be Determined (TBD)

1. Introduction

1.1 Purpose

This document specifies the software requirements for a Research Management Application (RMA) designed to assist researchers in managing the entire lifecycle of research projects, from proposal to publication. The system aims to streamline project administration, literature review, experiment tracking, collaboration, data analysis, and report generation.

1.2 Scope

The Research Management Application is a web-based application intended for use by individual researchers, research teams, and academic institutions. It provides tools for: - Creating and managing research projects - Organizing and annotating literature - Designing, tracking, and analyzing experiments - Collaborating with team members via shared workspaces and communication tools - Performing data analysis and generating visualizations - Automating report and manuscript preparation

The system will be accessible via web browsers and will have responsive design for tablet and mobile access. It will not include specialized laboratory equipment control or proprietary data analysis algorithms (these would be integrated via plugins or APIs).

1.3 Definitions, Acronyms, and Abbreviations

- **RMA**: Research Management Application
- **PI**: Principal Investigator
- **Co-I**: Co-Investigator
- **IRB**: Institutional Review Board
- **PICO**: Population, Intervention, Comparison, Outcome (for literature reviews)
- **API**: Application Programming Interface
- **UI**: User Interface
- **UX**: User Experience
- **API**: Application Programming Interface
- **REST**: Representational State Transfer
- **JSON**: JavaScript Object Notation

1.4 References

- [IEEE Std 830-1998] IEEE Recommended Practice for Software Requirements Specifications
- [ISO/IEC/IEEE 29148:2018] Systems and software engineering — Life cycle processes — Requirements engineering
- Internal Research Management Guidelines (Institution-specific, to be provided)
- [Markdown Guide](#)

1.5 Overview

Section 2 provides an overall description of the product, including product perspective, functions, user characteristics, constraints, and documentation. Section 3 details specific requirements, including external interfaces, functional and non-functional requirements, and database requirements. Section 4 elaborates on system features with detailed descriptions. Section 5 covers other non-functional requirements. Section 6 lists other requirements (legal, licensing, etc.). Appendices provide detailed models, including data flow diagrams, entity-relationship diagrams, use case diagrams, and activity diagrams.

2. Overall Description

2.1 Product Perspective

The Research Management Application is a self-contained web application designed to operate in a standard LAMP/WAPP environment (Linux/Windows, Apache, MySQL/MariaDB, PHP/Python/Perl) or equivalent (e.g., MEAN, Django). It will be deployed on institutional servers or cloud infrastructure (AWS, Azure, GCP) and accessed via standard web browsers (Chrome, Firefox, Safari, Edge). The system does not require integration with other proprietary systems but provides APIs for integration with reference managers (Zotero, Mendeley), electronic lab notebooks (LabArchives, Benchling), and data repositories (Figshare, Dryad).

2.2 Product Functions

The RMA provides the following core functions: 1. **Project Management**: Create, plan, and track research projects; manage milestones, tasks, and budgets. 2. **Literature Management**: Import, organize, annotate, and categorize scholarly articles; generate bibliographies. 3. **Experiment Tracking**: Design experiments, record protocols, log results, manage samples and reagents. 4. **Collaboration Tools**: Share projects, assign roles and permissions, communicate via comments and notifications, manage version control. 5. **Data Analysis and Visualization**: Import datasets, apply statistical tests, generate plots and charts, export results. 6. **Report Generation**: Create progress reports, IRB proposals, manuscripts, and presentations using templates.

2.3 User Classes and Characteristics

User Class	Description	Characteristics
Principal Investigator (PI)	Leads research projects, responsible for overall direction and funding	Requires overview dashboards, budget tracking, approval workflows, high-level reporting
Co-Investigator (Co-I)	Contributes to specific aspects of the project	Needs access to assigned tasks, data, and literature; can collaborate on experiments and writing
Research Assistant (RA)	Assists with day-to-day tasks, data collection, and literature review	Requires task lists, data entry forms, literature annotation tools, basic analysis
Student (Graduate/Undergraduate)	Assists under supervision, may be working on thesis/dissertation	Needs access to project resources, literature, and basic tools; limited administrative rights
Administrator	Manages system installation, user accounts, and institutional settings	Requires user management, security settings, backup/restore, and system monitoring
Collaborator (External)	External researcher invited to contribute to specific aspects	Limited access to shared components; controlled by PI permissions

2.4 Operating Environment

- **Hardware**: Server: Minimum 2 vCPU, 4GB RAM, 50GB SSD; Client: Any modern computer or tablet with browser
- **Operating System (Server)**: Linux (Ubuntu 20.04 LTS or later), Windows Server 2019+, or equivalent
- **Operating System (Client)**: Windows 10+, macOS 10.15+, Linux (Ubuntu 20.04+), iOS 13+, Android 10+
- **Software**:
 - Web Server: Apache 2.4+, Nginx 1.18+, or IIS 10+
 - Database: MySQL 8.0+, MariaDB 10.5+, or PostgreSQL 12+
 - Language: Python 3.8+ (Django/FastAPI) or Node.js 14+ (Express/NestJS)
 - Browser: Latest versions of Chrome, Firefox, Safari, Edge
 - Optional: LaTeX (for PDF report generation), R/Python libraries (for analysis)
- **Communication**: HTTP/HTTPS (port 443 for HTTPS), SMTP for email notifications

- **Security:** TLS 1.2+ for data in transit; AES-256 for data at rest (optional)

2.5 Design and Implementation Constraints

- Must be responsive and accessible (WCAG 2.1 AA compliant)
- Must comply with institutional data protection policies (e.g., FERPA, GDPR where applicable)
- Must support LDAP/Active Directory authentication for institutional deployments
- Must provide RESTful API for integration with external systems
- Must be deployable via Docker containers for ease of deployment
- Must support multi-tenancy for institutional deployments (optional, for SaaS model)
- Must use open-source libraries where possible to minimize licensing costs
- Must have offline capabilities for data entry (with sync when online) - [Optional, Phase 2]

2.6 User Documentation

- Online Help System: Context-sensitive help accessible via ? icon in each module
- User Manual: Comprehensive PDF and HTML guide covering all features
- Video Tutorials: Short (<5 min) videos demonstrating key workflows
- FAQ: Searchable knowledge base of common questions and troubleshooting
- API Documentation: Swagger/OpenAPI documentation for developers
- Administrator Guide: Installation, configuration, maintenance, and troubleshooting

2.7 Assumptions and Dependencies

- **Assumptions:**
 - Institutions provide adequate server infrastructure or cloud subscription
 - Users have basic computer literacy and familiarity with web applications
 - Research projects follow standard methodologies (experimental, observational, etc.)
 - Literature import will primarily use standard formats (RIS, BibTeX, PubMed XML)
 - Data files will be in common formats (CSV, Excel, JSON, common image formats)
- **Dependencies:**
 - Reliable internet connectivity for cloud-based features
 - Compatible web browser with JavaScript enabled
 - For PDF generation: LaTeX distribution (TeX Live, MikTeX) or headless Chrome
 - For advanced analysis: R or Python with necessary libraries (numpy, pandas, matplotlib, etc.)
 - Authentication system: LDAP/AD or internal user database

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 USER INTERFACES

- **Web Interface:** Responsive design using Bootstrap 5 or equivalent CSS framework
- **Navigation:** Persistent left-sidebar navigation with collapsible sections; top-bar for user profile and notifications
- **Forms:** Consistent form design with validation; use of date pickers, rich text editors (for notes), file upload widgets
- **Data Tables:** Sortable, searchable, paginated tables with bulk action capabilities
- **Dashboards:** Customizable widgets showing project status, upcoming deadlines, recent activity
- **Visualization:** Interactive plots using Plotly.js or Chart.js; ability to export as PNG, SVG, PDF
- **Accessibility:** ARIA labels, keyboard navigation, sufficient color contrast, screen reader friendly

3.1.2 HARDWARE INTERFACES

- Standard input devices: keyboard, mouse, touchpad, touchscreen
- Optional: Barcode scanner for sample/inventory management (via USB or Bluetooth)
- Optional: Webcam for document scanning (via browser getUserMedia API)

3.1.3 SOFTWARE INTERFACES

- **Authentication:** LDAP/Active Directory, OAuth 2.0 (Google, ORCID), local database
- **Storage:** Local file system (for uploads), optional cloud storage (AWS S3, Azure Blob) via plugins
- **Analysis Tools:**
 - Rserve or rpy2 for R integration
 - Jupyter kernels for Python/R integration (optional, via plugins)
 - API endpoints for external analysis tools
- **Reference Managers:**
 - Zotero API
 - Mendeley API
 - Crossref API for metadata lookup
 - PubMed API for literature search
- **Data Repositories:**
 - Figshare API
 - Dryad API
 - Institutional repository APIs for publishing data
- **Communication:**
 - SMTP for email notifications
 - WebSocket for real-time notifications (optional)
 - Slack/Microsoft Teams webhooks for notifications (optional)

3.1.4 COMMUNICATIONS INTERFACES

- **Primary:** HTTP/HTTPS (RESTful API)
- **Real-time Updates:** WebSocket (optional, for live collaboration features)
- **Email:** SMTP for notifications and reports
- **File Transfer:** HTTP(S) for upload/download; optional FTP/SFTP for large batches

3.2 Functional Requirements

3.2.1 PROJECT MANAGEMENT

ID	Requirement	Priority	Notes
PM1	Create a new research project with title, abstract, start/end dates, PI, and Co-Is	High	Must support multiple PIs
PM2	Define project objectives, hypotheses, and milestones	High	Milestones can have dependencies
PM3	Create and assign tasks to team members with due dates and priority levels	High	Tasks can be linked to milestones
PM4	Track project budget: allocate funds, record expenses, generate budget reports	Medium	Supports multiple funding sources
PM5	Generate Gantt chart view of project timeline and task dependencies	Medium	Using JavaScript Gantt library
PM6	Store and manage project documents (proposals, IRB protocols, consent forms)	High	Version control for documents
PM7	Set up approval workflows for documents and milestones (e.g., IRB approval required before participant recruitment)	Medium	Configurable workflow steps
PM8	View project dashboard showing progress, upcoming deadlines, budget status, and recent activity	High	Customizable widgets

3.2.2 LITERATURE MANAGEMENT

ID	Requirement	Priority	Notes
LM1	Import references from RIS, BibTeX, PubMed XML, or manual entry	High	Batch import supported
LM2	Search internal library by title, author, year, keywords, tags	High	Full-text search if PDF text extracted
LM3	Attach and store PDFs of articles (with copyright compliance notice)	High	Storage limits configurable
LM4	Annotate PDFs with highlights, notes, and tags (using PDF.js or similar)	Medium	Annotations stored separately
LM5	Organize references into collections, tags, and custom fields	High	Supports hierarchical tags

ID	Requirement	Priority	Notes
LM6	Generate bibliographies in multiple styles (APA, MLA, Vancouver, Chicago, etc.)	High	Using CiteProc-js or similar
LM7	Detect and manage duplicate references	Medium	Based on DOI, PMID, or title/author matching
LM8	Sync with external reference managers (Zotero, Mendeley) via their APIs	Low	Future enhancement
LM9	Literature review workspace: build tables (e.g., PICO tables) and synthesize findings	Medium	Spreadsheet-like interface

3.2.3 EXPERIMENT TRACKING

ID	Requirement	Priority	Notes
ET1	Create experiment protocols with steps, materials, and safety notes	High	Protocols can be templated and reused
ET2	Record experimental runs: date, operator, conditions, observations, results	High	Structured data entry with validation
ET3	Manage inventory of samples, reagents, and equipment	Medium	Track location, quantity, expiration dates
ET4	Link experimental runs to specific protocols and project milestones	High	Enables traceability
ET5	Record deviations from protocol and corrective actions	Medium	Important for reproducibility
ET6	Store and link raw data files (images, sequences, measurements) to experimental runs	High	Supports versioning of data files
ET7	Generate experimental logs and reports in PDF/HTML format	Medium	Customizable templates
ET8	Barcode/QR code scanning for sample and reagent tracking (optional)	Low	Requires hardware integration
ET9	Collaborative protocol editing with version history and commenting	Medium	Similar to Google Docs for protocols

3.2.4 COLLABORATION TOOLS

ID	Requirement	Priority	Notes
COL1	Invite team members by email or username; assign roles (PI, Co-I, RA, Student, Collaborator)	High	Role-based access control (RBAC)
COL2	Define custom roles with granular permissions (create, read, update, delete) for each module	High	Permissions inherit from role but can be overridden per project
COL3	Comment on projects, experiments, literature, and documents; @mention users for notifications	High	Email and in-app notifications
COL4	Real-time co-editing of documents (optional, requires operational transform or CRDT)	Low	Future enhancement

ID	Requirement	Priority	Notes
COL5	Activity feed showing recent changes across projects the user is involved in	High	Filterable by project, user, and activity type
COL6	Task assignment and tracking with notifications for due dates and status changes	High	Integrated with project management tasks
COL7	Discussion forums for project-wide communication (separate from item-specific comments)	Medium	Threaded discussions
COL8	Version control for documents and protocols: view history, revert to previous versions	High	Basic diff-view for text files
COL9	Notify users via email and in-app alerts for: mentions, task assignments, deadlines, document updates	High	Configurable notification preferences

3.2.5 DATA ANALYSIS AND VISUALIZATION

ID	Requirement	Priority	Notes
DAV1	Import data from CSV, Excel (XLS/XLSX), JSON, SQL query, or direct database connection	High	Specify delimiters, encoding, and data types
DAV2	Preview and clean data: handle missing values, outliers, data type conversion	Medium	Basic data wrangling capabilities
DAV3	Perform descriptive statistics: mean, median, std, min, max, frequency counts	High	For numeric and categorical data
DAV4	Generate common plots: scatter, line, bar, box, histogram, heatmap, pie chart	High	Using Plotly.js or Chart.js; interactive (zoom, pan, hover)
DAV5	Perform basic statistical tests: t-test, ANOVA, chi-square, correlation	Medium	Using built-in libraries (e.g., scipy via API or integrated R)
DAV6	Save analysis workflows (steps and parameters) for reproducibility and reuse	Medium	Similar to Jupyter notebooks or Galaxy workflows
DAV7	Export analysis results: plots (PNG, SVG, PDF), tables (CSV, Excel), reports (HTML, PDF)	High	Reports can include narrative and visualizations
DAV8	Integrate with external analysis tools: RStudio, Jupyter Notebook, Python scripts via API	Low	Launch external tools with context (dataset, variables)
DAV9	Version control for datasets: track changes to raw and processed data	Medium	Basic versioning; diff for CSV may be limited

3.2.6 REPORT GENERATION

ID	Requirement	Priority	Notes
RG1	Create documents using templates: project progress report, IRB proposal, manuscript, poster, presentation	High	Template variables pulled from project/data
RG2	Support for Markdown and LaTeX in templates for rich formatting	High	Pandoc or LaTeX engine for PDF generation

ID	Requirement	Priority	Notes
RG3	Drag-and-drop interface to insert tables, figures, citations, and text blocks into reports	Medium	WYSIWYG editor for template-based report building
RG4	Automatic bibliography generation from linked literature library	High	Uses same bibliography engine as LM6
RG5	Version control for reports: track changes and compare versions	Medium	Similar to document version control
RG6	Schedule automatic report generation and email distribution (e.g., monthly progress reports)	Low	Requires background job scheduler (cron)
RG7	Collaborative report writing with commenting and version history	Medium	Leverages collaboration tools
RG8	Export reports to multiple formats: HTML, PDF, DOCX, PPTX	High	Using Pandoc or similar conversion tools

3.3 Non-functional Requirements

3.3.1 PERFORMANCE

ID	Requirement	Priority	Notes
PF1	Page load time < 3 seconds for dashboard and main views on standard broadband	High	Measured with Lighthouse
PF2	Submit form and update data < 2 seconds for typical operations	High	AJAX requests without full page reload
PF3	Handle concurrent users: 50 simultaneous active users without degradation	Medium	For small team deployment
PF4	Generate reports with up to 100 pages in < 10 seconds	Medium	Depends on template complexity and data size
PF5	Search returns results in < 2 seconds for libraries up to 10,000 references	Medium	Full-text search may require external indexer (e.g., Elasticsearch)

3.3.2 SAFETY

ID	Requirement	Priority	Notes
SF1	System shall not modify or delete user data without explicit confirmation	High	Delete actions require confirmation dialog
SF2	Automatic save of work-in-progress every 2 minutes to prevent data loss	Medium	Local storage backup with server sync
SF3	Validate critical data entries (e.g., reagent concentrations, dosage amounts)	Medium	Custom validation rules per field type
SF4	Provide undo/redone capabilities for critical actions (experimental log entries, document edits)	Low	Limited to session-based undo for web storage

3.3.3 SECURITY

ID	Requirement	Priority	Notes
SEC1	All data transmission via HTTPS (TLS 1.2+)	High	Enforced at web server level
SEC2	Passwords hashed using bcrypt or Argon2 with salt	High	Industry-standard password storage
SEC3	Role-based access control (RBAC) enforced on all resources	High	Users can only access resources permitted by their role
SEC4	Audit trail for all create, update, delete operations on critical data (projects, experiments, documents)	High	Logs: user, timestamp, action, object ID, changes
SEC5	Session timeout after 30 minutes of inactivity	Medium	Extendable with user activity
SEC6	Prevent common web vulnerabilities: SQL injection, XSS, CSRF	High	Using framework protections (Django middleware, Express helmet, etc.)
SEC7	Data encryption at rest for sensitive fields (SSN, DOB, etc.) if stored	Medium	AES-256; configurable per field type
SEC8	Support for data export restrictions: prevent downloading of sensitive data by role	Medium	Based on data classification tags
SEC9	Regular security audits and penetration testing (outsourced or using open-source tools)	Low	Organizational responsibility

3.3.4 SOFTWARE QUALITY ATTRIBUTES

ID	Requirement	Priority	Notes
QA1	Modular architecture: clear separation between presentation, business logic, and data layers	High	Enables maintainability and testing
QA2	Code documentation: all public methods and classes have docstrings/comments	Medium	Generated documentation via Sphinx/JSDoc
QA3	Automated test coverage: unit tests for >80% of backend logic	Medium	Using pytest/Jest
QA4	Internationalization (i18n) support for UI labels and messages	Low	Future enhancement for non-English deployments
QA5	Logging: structured logs (JSON) for debugging and monitoring	Medium	Log levels: DEBUG, INFO, WARN, ERROR
QA6	Error handling: user-friendly error messages without exposing stack traces	High	Log detailed errors server-side
QA7	Backup and restore functionality: automated database and file backups	Medium	Configurable schedule (daily/weekly)
QA8	Scalability: horizontal scaling possible via load balancer and shared database/storage	Low	For large institutional deployments

3.4 Database Requirements

- **Primary Database:** Relational (PostgreSQL/MySQL) for structured data (users, projects, experiments, literature references)
- **File Storage:** Filesystem or object storage (AWS S3) for uploaded files (PDFs, images, data files)
- **Search Index:** Optional external search engine (Elasticsearch) for full-text search on literature and documents
- **Backup Strategy:**
 - Daily incremental backups
 - Weekly full backups
 - Retention: 30 days for daily, 12 weeks for weekly
 - Off-site storage recommended
- **Data Retention:**
 - Raw data: retained per institutional policy (default 7 years post-publication)
 - Processed data and analysis: retained as long as project is active + 2 years
 - Anonymized data for retention: may be kept longer for meta-research
- **Data Portability:**
 - Export functions for all user data in standard formats (CSV, JSON, XML, ZIP)
 - GDPR-compliant data deletion upon request

4. System Features

This section provides detailed descriptions of each major feature, including inputs, outputs, functionality, and relevant diagrams described in text.

4.1 Project Management

DESCRIPTION AND PRIORITY

The Project Management module allows users to create, plan, and monitor research projects from initiation to completion. It provides tools for defining scope, setting milestones, assigning tasks, tracking budgets, and monitoring progress. **Priority: High**

STIMULUS/RESPONSE SEQUENCES

- **Create Project:**
 - User clicks "New Project" → fills form (title, abstract, dates, PI, Co-Is, funding source) → clicks "Create" → system validates input, creates project record, assigns ID, redirects to project dashboard.
- **Add Milestone:**
 - In project view, user clicks "Add Milestone" → enters description, due date, optional dependencies → system validates, creates milestone, updates Gantt chart.
- **Assign Task:**

- User opens task module → clicks "New Task" → selects project, assignee, sets description, due date, priority → system creates task, sends notification to assignee, appears in assignee's task list.
- **Track Budget:**
- User navigates to budget tab → adds line item (description, amount, category, date) → system updates total, compares to allocated budget, shows variance.
- **View Dashboard:**
- User visits project dashboard → system aggregates data: % tasks complete, upcoming milestones, budget spent/recent expenses, recent activity → displays widgets.

FUNCTIONAL REQUIREMENTS

- PM1-PM8 as listed in Section 3.2.1

DESIGN CONSTRAINTS

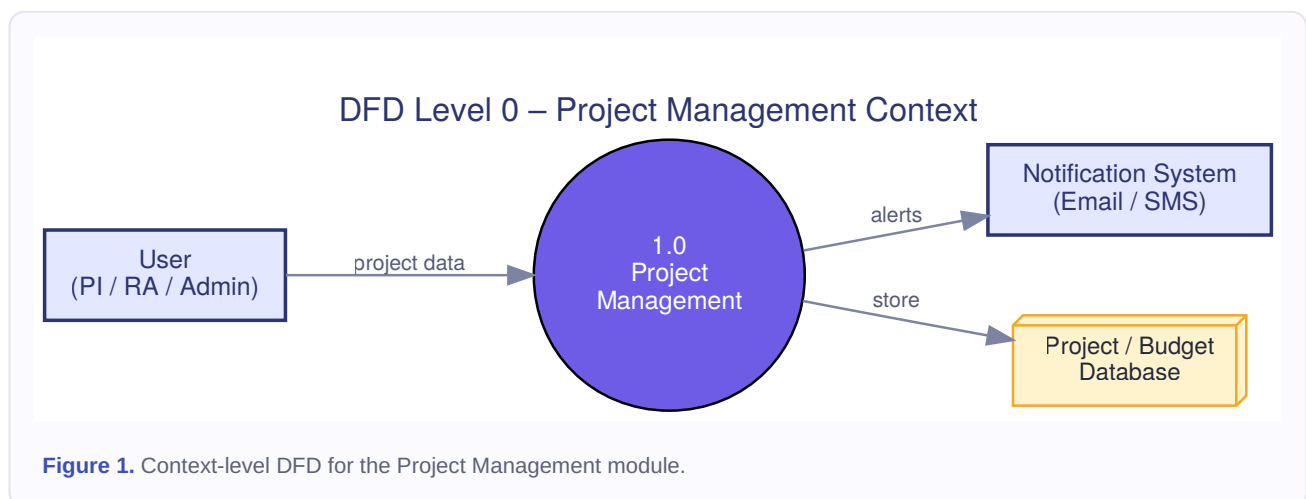
- Must use responsive design for dashboard widgets
- Gantt chart must be interactive (drag to adjust dates, zoom)
- Budget calculations must handle multiple currencies with exchange rates (if enabled)

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, SF1, SF2, SEC3, SEC4, QA1, QA7

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Project Management Context



Shows the Project Management module interacting with external entities: User (Researcher, PI, Admin), Database (Project Data, User Data, Financial Data), and Notification System (Email/SMS).

Data Flow Diagram (DFD) Level 1 - Maintain Project Information

Processes: 1.0 Validate Project Data, 2.0 Store Project Information, 3.0 Calculate Progress Metrics, 4.0 Generate Notifications.

Data Stores: Project Master, Milestone Table, Task Table, Budget Table, User Preferences.

External Entities: User, Notification Service.

Entity-Relationship Diagram (ERD) Snippet - Project Management

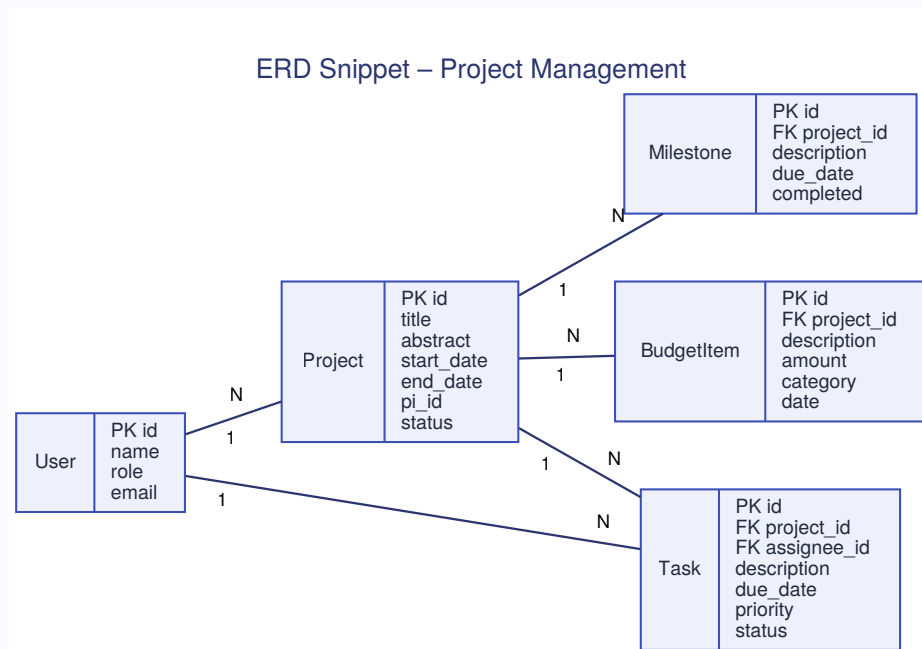


Figure 2. Core entity-relationship model for Project Management.

Entities: Project (id, title, abstract, start_date, end_date, pi_id, status), Milestone (id, project_id, description, due_date, completed), Task (id, project_id, assignee_id, description, due_date, priority, status), BudgetItem (id, project_id, description, amount, category, date).

Relationships: Project "has many" Milestone, Task, BudgetItem; User "assigned to" Task; Project "has one" PI (User).

Use Case Diagram Snippet - Project Management

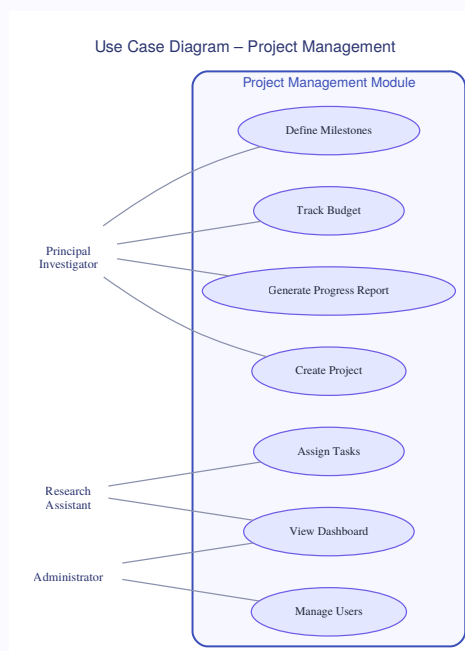


Figure 3. Actors and use cases for the Project Management module.

Actors: Principal Investigator, Research Assistant, Administrator.

Use Cases: Create Project, Define Milestones, Assign Tasks, Track Budget, View Dashboard, Generate Progress Report.

Relationships: PI "generalizes" User; RA "generalizes" User; Admin "includes" Manage Users.

4.2 Literature Management

DESCRIPTION AND PRIORITY

The Literature Management module enables researchers to import, organize, annotate, and cite scholarly works. It supports building literature reviews, managing PDFs, and generating bibliographies in various styles. **Priority: High**

STIMULUS/RESPONSE SEQUENCES

- **Import References:**
 - User clicks "Import References" → selects file (RIS/BibTeX) or enters search query (PubMed) → system parses input, checks for duplicates, imports references into library → shows import summary with counts of new/duplicates.
- **Annotate PDF:**
 - User opens reference → clicks "Annotate PDF" → PDF viewer loads with annotation tools (highlight, note, tag) → user makes annotations → system saves annotations linked to reference and user ID.
- **Generate Bibliography:**
 - User selects references → chooses citation style → clicks "Generate Bibliography" → system formats references using CiteProc-js → displays formatted list; user can copy or export as RTF/HTML.
- **Search Library:**
 - User enters search terms in search bar → system searches title, author, abstract, tags, notes → returns ranked results with highlights → user can filter by year, tags, collections.

FUNCTIONAL REQUIREMENTS

- LM1-LM9 as listed in Section 3.2.2

DESIGN CONSTRAINTS

- PDF annotation must work in modern browsers without plugins (using PDF.js or similar)
- Bibliography generation must support CSL (Citation Style Language) styles
- Duplicate detection must be configurable (exact match, fuzzy match on title/author)

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, PF5, SF1, SF3, SEC3, SEC4, QA1, QA2, QA3

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Literature Management Context

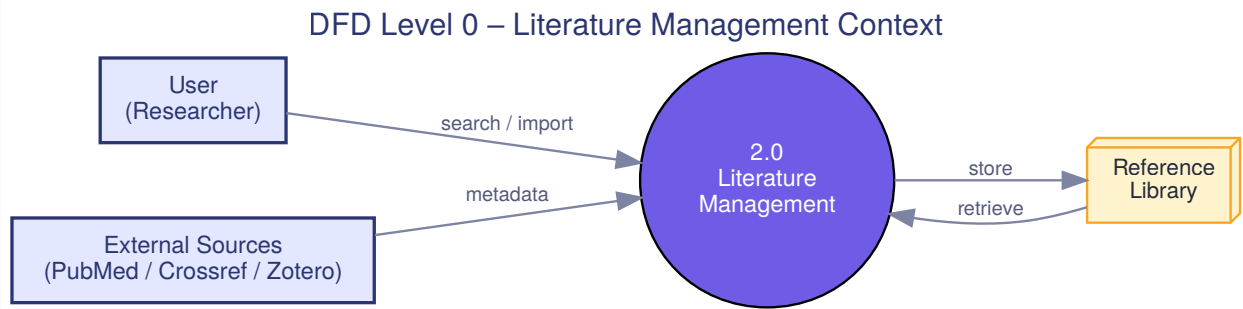


Figure 4. Context-level DFD for the Literature Management module.

Shows interaction with: User, External Sources (PubMed, Crossref, Zotero API), Document Storage (File System/Cloud), Citation Style Repository.

Data Flow Diagram (DFD) Level 1 - Import and Organize References

Processes: 1.0 Validate Input Format, 2.0 Parse References, 3.0 Detect Duplicates, 4.0 Store References, 5.0 Update Index.

Data Stores: Reference Library, Duplicate Log, Search Index.

Entity-Relationship Diagram (ERD) Snippet - Literature Management

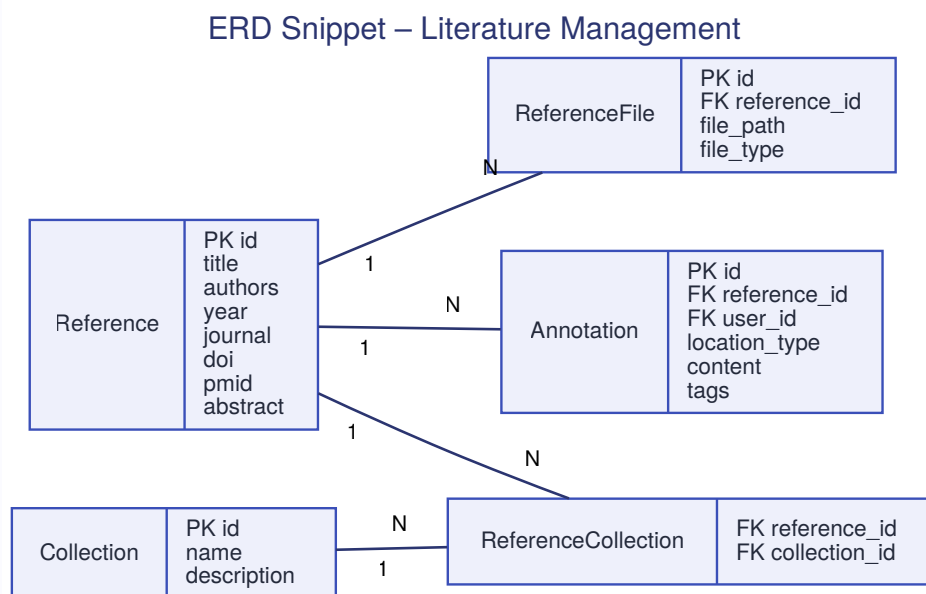


Figure 5. Core entity-relationship model for Literature Management.

Entities: Reference (id, title, authors, year, journal, volume, pages, doi, pmid, url, abstract), ReferenceFile (id, reference_id, file_path, file_type), Annotation (id, reference_id, user_id, location_type, content, tags), Collection (id, name, description), ReferenceCollection (id, reference_id, collection_id).

Relationships: Reference "has many" ReferenceFile and Annotation; Collection "has many" Reference via ReferenceCollection.

Use Case Diagram Snippet - Literature Management

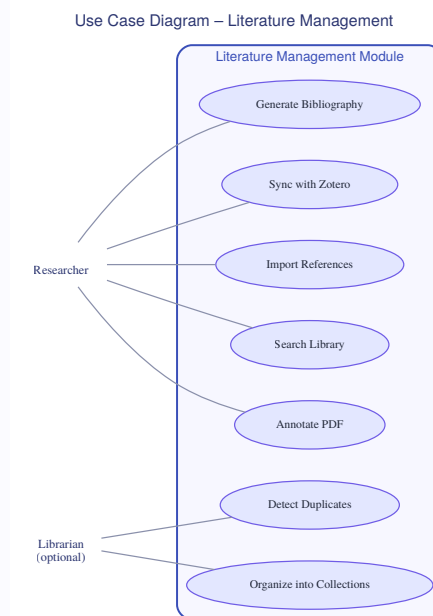


Figure 6. Actors and use cases for the Literature Management module.

Actors: Researcher, Librarian (optional).

Use Cases: Import References, Search Library, Annotate PDF, Organize into Collections, Generate Bibliography, Detect Duplicates, Sync with Zotero.

Relationships: Researcher "includes" Annotate PDF; Librarian "extends" Manage Collections.

4.3 Experiment Tracking

DESCRIPTION AND PRIORITY

The Experiment Tracking module supports the documentation, execution, and analysis of laboratory experiments. It ensures reproducibility by capturing detailed protocols, conditions, results, and deviations.

Priority: High

STIMULUS/RESPONSE SEQUENCES

- **Create Protocol:**
 - User clicks "New Protocol" → enters title, objective, steps, materials, safety notes → system saves protocol version 1.0 → user can use template from library.
- **Run Experiment:**
 - User selects protocol → clicks "New Run" → enters date, operator, conditions (temperature, pH, etc.) → for each step, user records observations, measurements, notes → system saves run linked to protocol and project.
- **Record Deviations:**
 - During experiment run, user flags a step as deviation → describes what happened and corrective action → system logs deviation with timestamp.

- **Link Data Files:**

- User uploads data file (image, CSV, etc.) → system stores file, extracts metadata if possible → links file to specific experiment run or protocol step.

- **Generate Experiment Report:**

- User selects one or more experiment runs → chooses template (e.g., lab notebook entry) → system populates template with protocol, conditions, results, deviations → generates PDF/HTML.

FUNCTIONAL REQUIREMENTS

- ET1-ET9 as listed in Section 3.2.3

DESIGN CONSTRAINTS

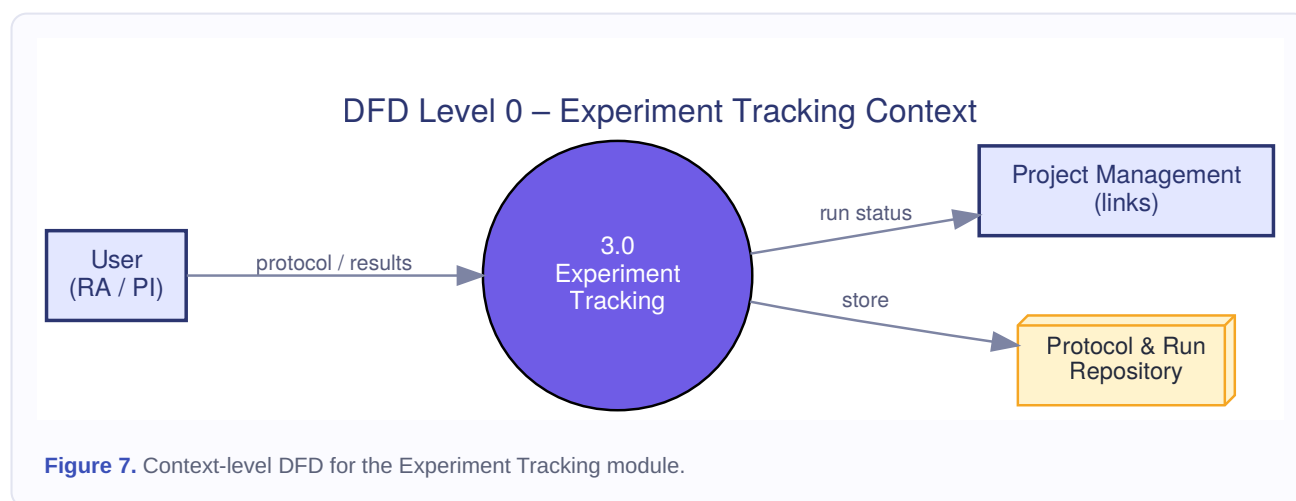
- Protocol steps must be structured (ordered list) to allow indexing and deviation tracking
- Data file linking must support versioning (e.g., raw vs. processed)
- Deviation tracking must be mandatory for regulated research (configurable)

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, SF1, SF2, SF3, SF4, SEC3, SEC4, QA1, QA6

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Experiment Tracking Context



Shows interaction with: User, Laboratory Equipment (via manual entry), Data Storage (files), Project Management (links to projects/milestones).

Data Flow Diagram (DFD) Level 1 - Conduct Experiment

Processes: 1.0 Select Protocol, 2.0 Initialize Run, 3.0 Record Step Data, 4.0 Check for Deviations, 5.0 Store Results, 6.0 Link Data Files.

Data Stores: Protocol Library, Experiment Runs, Step Data, Deviations, File Repository.

Entity-Relationship Diagram (ERD) Snippet - Experiment Tracking

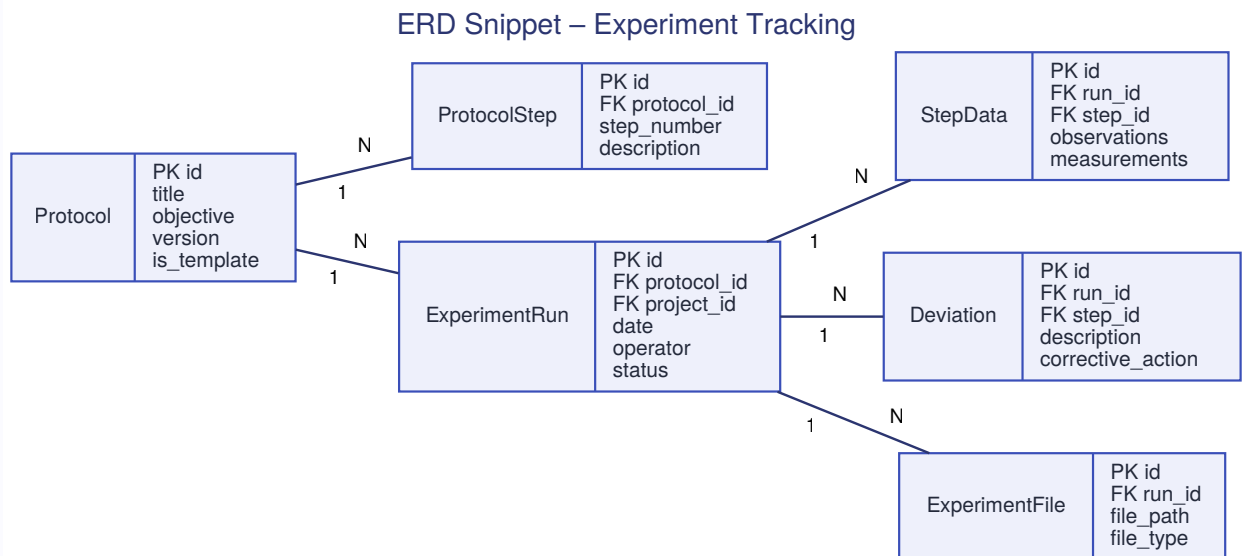


Figure 8. Core entity-relationship model for Experiment Tracking.

Entities: Protocol (id, title, objective, version, is_template), ProtocolStep (id, protocol_id, step_number, description, materials), ExperimentRun (id, protocol_id, project_id, date, operator, status), StepData (id, run_id, step_id, observations, measurements, notes), Deviation (id, run_id, step_id, description, corrective_action, timestamp), ExperimentFile (id, run_id, file_path, file_type, description).

Relationships: Protocol "has many" ProtocolStep; ExperimentRun "has many" StepData and Deviation and ExperimentFile; Project "has many" ExperimentRun.

Use Case Diagram Snippet - Experiment Tracking

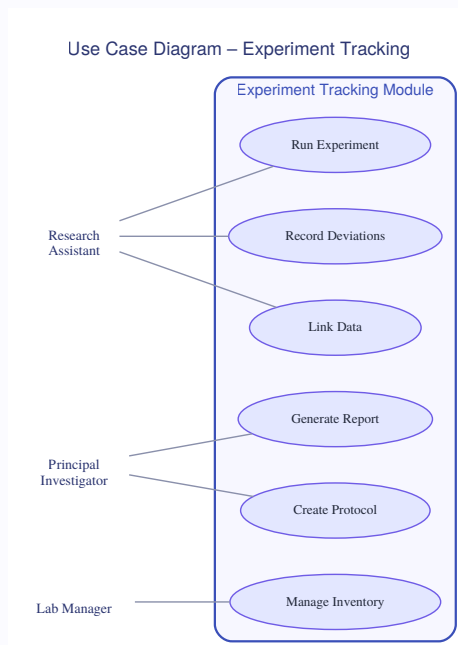


Figure 9. Actors and use cases for the Experiment Tracking module.

Actors: Research Assistant, Principal Investigator, Lab Manager.

Use Cases: Create Protocol, Run Experiment, Record Deviations, Link Data, Generate Report, Manage Inventory.

Relationships: Lab Manager "extends" Manage Inventory; PI "authorizes" ExperimentRun (for high-risk experiments).

4.4 Collaboration Tools

DESCRIPTION AND PRIORITY

The Collaboration Tools module facilitates communication, task coordination, and version-controlled document sharing among research team members. It ensures that all contributors stay informed and can work together effectively. **Priority: High**

STIMULUS/RESPONSE SEQUENCES

- **Invite Team Member:**
 - PI or Admin clicks "Invite Member" → enters email/username, selects role, selects projects to grant access → system sends invitation email with link; upon acceptance, user is added with specified role.
- **Comment on Item:**
 - User views an experiment run → clicks "Add Comment" → types message, optionally @mentions another user → system saves comment, sends email and in-app notification to mentioned users and item followers.
- **Assign Task:**
 - User views task list → clicks "Assign Task" → selects user, sets due date, priority → system creates task assignment, notifies assignee, appears in their "My Tasks" list.
- **Update Document:**
 - User opens document → makes changes → system auto-saves draft every 2 minutes → on explicit save, creates new version, shows diff from previous version.
- **View Activity Feed:**
 - User navigates to activity page → system fetches recent actions from projects user is involved in → displays sorted by time, with filters for project, user, action type.

FUNCTIONAL REQUIREMENTS

- COL1-COL9 as listed in Section 3.2.4

DESIGN CONSTRAINTS

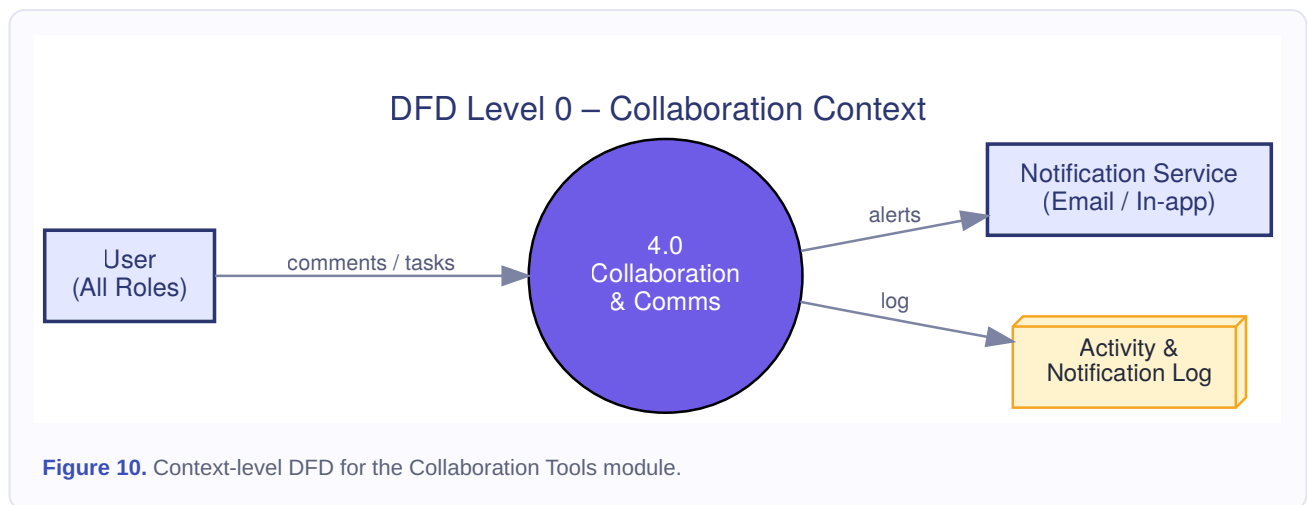
- Comments must support basic Markdown formatting
- Notification preferences must be granular (per project, per action type)
- Version differences should be visible via side-by-side diff for text files

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, SF1, SF2, SF3, SF4, SEC3, SEC4, QA1, QA5

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Collaboration Context



Shows interaction with: User, Notification Service (Email, In-app), Document Storage, Project Data, Task Data.

Data Flow Diagram (DFD) Level 1 - Manage Communication

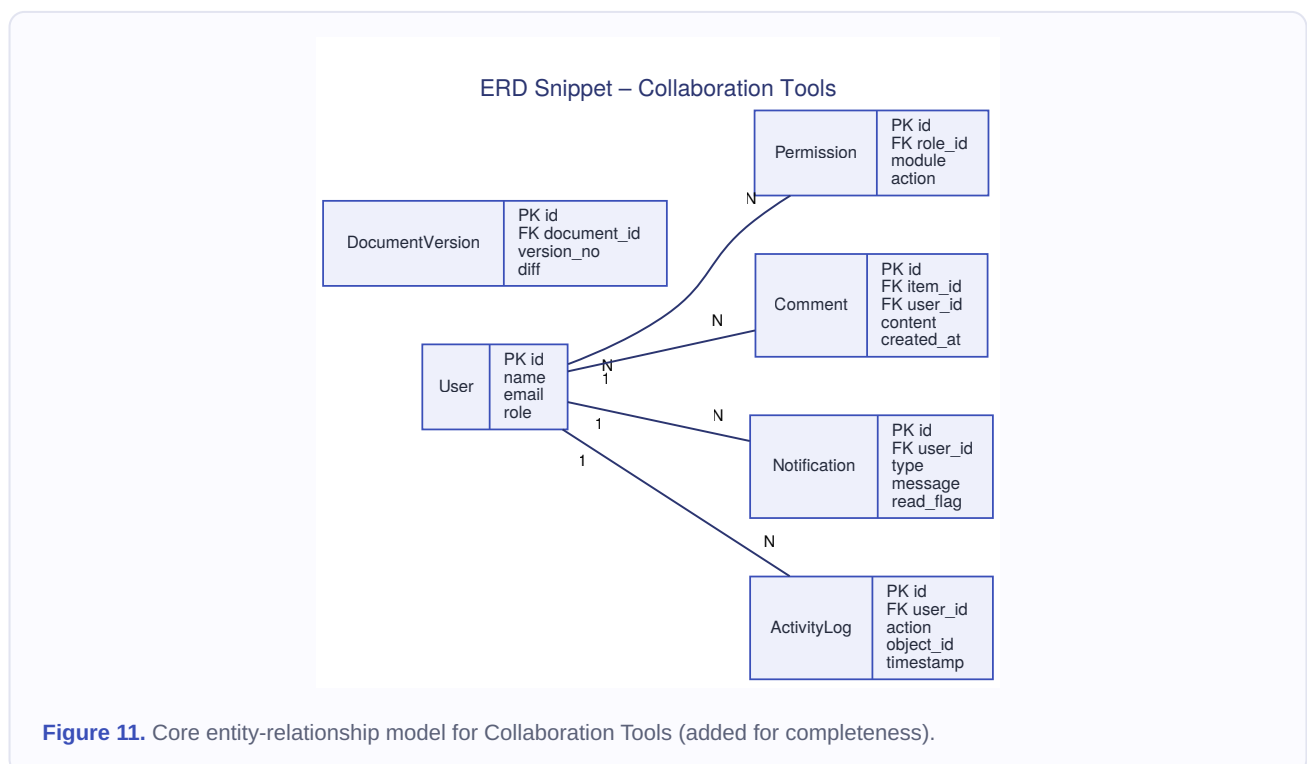
Processes: 1.0 Receive User Action, 2.0 Determine Recipients, 3.0 Create Notification, 4.0 Send Notification, 5.0 Update Activity Log.

Data Stores: User Preferences, Notification Queue, Activity Log.

Entity-Relationship Diagram (ERD) Snippet - Collaboration Tools

Entities: User, Permission, Comment, Notification, ActivityLog, DocumentVersion.

Relationships: User "has many" Comment, Notification, ActivityLog; User "has many" Permission (via Role).



Use Case Diagram Snippet - Collaboration

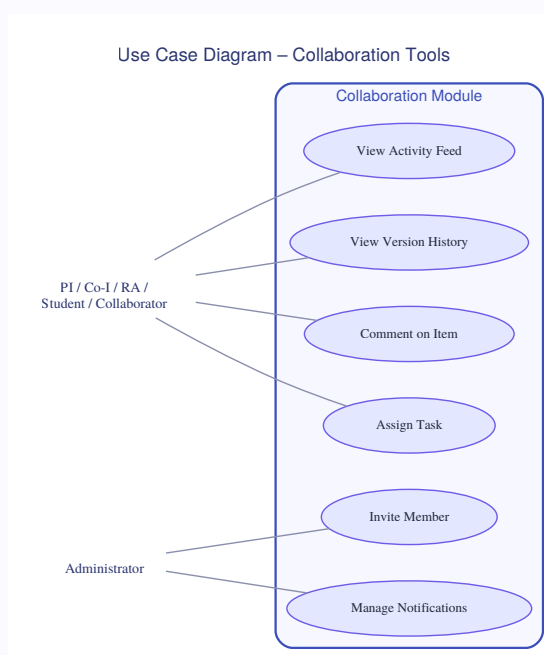


Figure 12. Actors and use cases for the Collaboration Tools module.

Actors: All User Types (PI, Co-I, RA, Student, Collaborator, Admin).

Use Cases: Invite Member, Comment on Item, Assign Task, View Activity Feed, Manage Notifications, View Version History.

Relationships: All actors "use" Comment on Item; Admin "manages" Notification Settings globally.

4.5 Data Analysis and Visualization

DESCRIPTION AND PRIORITY

The Data Analysis and Visualization module provides tools for importing, cleaning, analyzing, and visualizing research data. It supports basic statistical tests and common plot types, with export options for further processing or publication. **Priority: Medium**

STIMULUS/RESPONSE SEQUENCES

• Import Data:

- User clicks "Import Data" → selects file (CSV, Excel) or connects to database → system previews first few rows, asks for delimiter/encoding → user confirms column types → system imports data into temporary dataset.

• Clean Data:

- User selects dataset → opens cleaning wizard → handles missing values (remove/impute), outliers (winsorize/remove), data types → system applies transformations, shows summary statistics.

• Create Visualization:

- User selects dataset → chooses chart type (scatter, bar, etc.) → maps variables to axes, color, size → system renders interactive chart → user can zoom, pan, hover for tooltips, adjust aesthetics.

- **Run Statistical Test:**
- User selects test type (t-test, ANOVA) → selects variables/groups → system checks assumptions (normality, equal variance) → runs test → displays p-value, confidence interval, effect size.
- **Export Results:**
- User selects chart or test result → chooses format (PNG, SVG, PDF for charts; CSV, Excel for tables) → system generates file → download initiates.

FUNCTIONAL REQUIREMENTS

- DAV1-DAV9 as listed in Section 3.2.5

DESIGN CONSTRAINTS

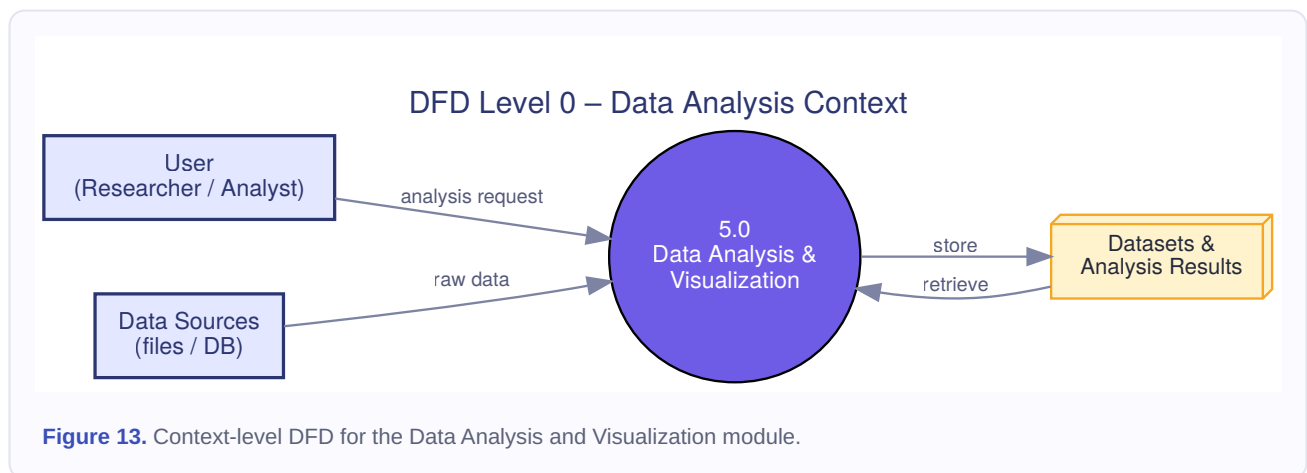
- Visualizations must be responsive and work on mobile devices
- Statistical tests must use well-established libraries (e.g., SciPy, R) and report assumptions
- Exported files must maintain quality (vector formats for plots, high-resolution raster)

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, PF3, PF4, SF1, SF2, SEC3, SEC4, QA1, QA6

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Data Analysis Context



Shows interaction with: User, Data Sources (files, databases), Analysis Engines (local or remote), Storage (results, plots).

Data Flow Diagram (DFD) Level 1 - Analyze Dataset

Processes: 1.0 Ingest Data, 2.0 Profile Data, 3.0 Clean/Transform Data, 4.0 Analyze Data, 5.0 Visualize Results, 6.0 Store Outputs.

Data Stores: Raw Dataset, Processed Dataset, Analysis Results, Generated Plots.

Entity-Relationship Diagram (ERD) Snippet - Data Analysis

ERD Snippet – Data Analysis and Visualization

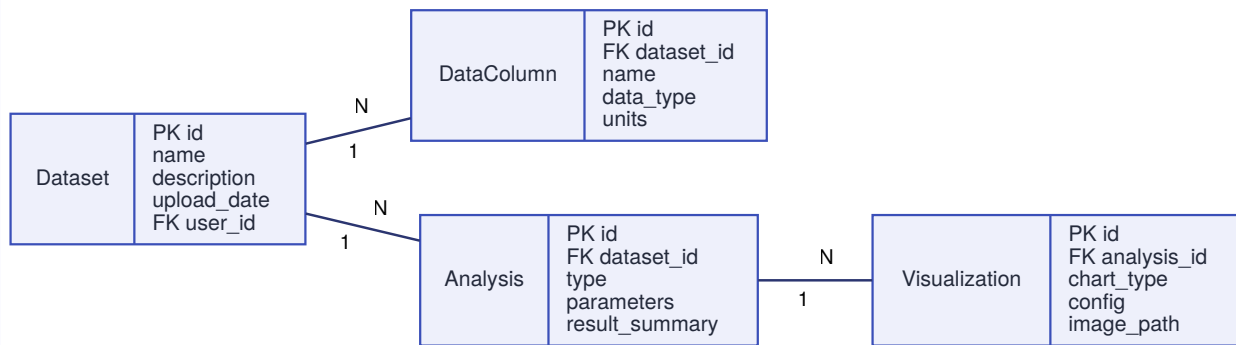


Figure 14. Core entity-relationship model for Data Analysis and Visualization.

Entities: Dataset (*id*, *name*, *description*, *upload_date*, *user_id*), DataColumn (*id*, *dataset_id*, *name*, *data_type*, *units*), Analysis (*id*, *dataset_id*, *type*, *parameters*, *result_summary*, *timestamp*), Visualization (*id*, *analysis_id*, *chart_type*, *config*, *image_path*).

Relationships: Dataset "has many" DataColumn; Analysis "based on" Dataset; Visualization "generated from" Analysis.

Use Case Diagram Snippet - Data Analysis

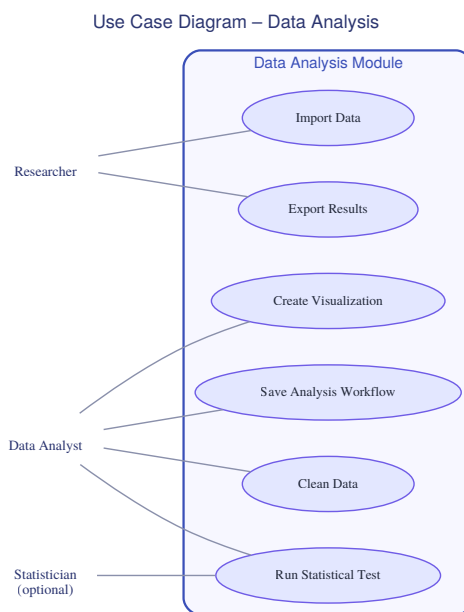


Figure 15. Actors and use cases for the Data Analysis module.

Actors: Researcher, Data Analyst, Statistician (optional).

Use Cases: Import Data, Clean Data, Create Visualization, Run Statistical Test, Export Results, Save Analysis Workflow.

Relationships: Statistician "extends" Run Statistical Test; Data Analyst "includes" Create Visualization and Run Statistical Test.

4.6 Report Generation

DESCRIPTION AND PRIORITY

The Report Generation module automates the creation of progress reports, grant proposals, manuscripts, and presentations by combining template-based writing with dynamic data from the project, literature, and experiments. **Priority: High**

STIMULUS/RESPONSE SEQUENCES

- **Create Report from Template:**
 - User clicks "New Report" → selects template (progress report, manuscript, etc.) → system creates document with placeholder fields → user fills in sections, inserts dynamic fields (e.g., {{project.title}}, {{budget.spent}}).
- **Insert Dynamic Content:**
 - While editing document, user clicks "Insert Field" → selects data source (project, experiment, literature) → chooses specific item and field (e.g., latest experiment result) → system inserts field code.
- **Insert Bibliography:**
 - User places cursor where bibliography should go → clicks "Insert Bibliography" → selects collection or tag → system generates formatted bibliography from selected references.
- **Generate PDF:**
 - User clicks "Export as PDF" → system processes document through template engine (e.g., Jinja2) and then through Pandoc/LaTeX → PDF generated → download initiates.
- **Schedule Report:**
 - User clicks "Schedule Report" → sets frequency (weekly, monthly), template, recipients → system creates scheduled job → at scheduled time, generates report and emails to recipients.

FUNCTIONAL REQUIREMENTS

- RG1-RG8 as listed in Section 3.2.6

DESIGN CONSTRAINTS

- Template language must be secure (sandboxed) to prevent code injection
- PDF generation must handle Unicode, images, and complex layouts
- Scheduled reports must be resilient to server restarts (use persistent job queue)

RELATED NON-FUNCTIONAL REQUIREMENTS

- PF1, PF2, PF4, SF1, SF2, SF3, SF5, SEC3, SEC4, QA1, QA7

DIAGRAMS DESCRIBED

Data Flow Diagram (DFD) Level 0 - Report Generation Context

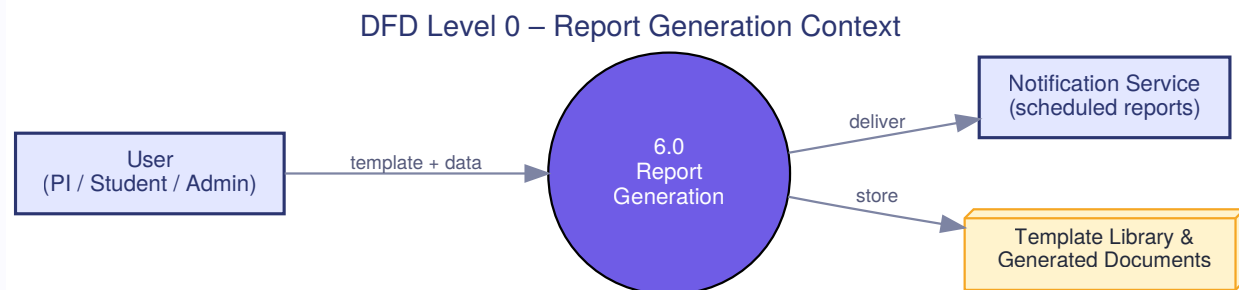


Figure 16. Context-level DFD for the Report Generation module.

Shows interaction with: User, Template Library, Project Data, Literature Data, Experiment Data, Storage (output documents), Notification Service (for scheduled reports).

Data Flow Diagram (DFD) Level 1 - Generate Report

Processes: 1.0 Select Template, 2.0 Merge Data, 3.0 Process Bibliography, 4.0 Generate Intermediate Format (HTML/Markdown), 5.0 Convert to Target Format (PDF/DOCX), 6.0 Deliver Result.

Data Stores: Template Library, Generated Documents.

Entity-Relationship Diagram (ERD) Snippet - Report Generation

Entities: Template, GeneratedDocument, Citation, Section.

Relationships: Template "has many" GeneratedDocument; GeneratedDocument "has many" Section and Citation.

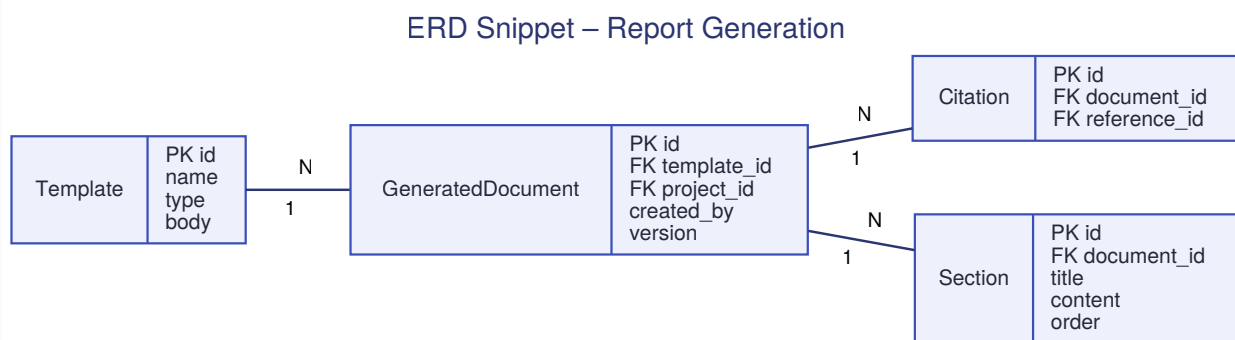


Figure 17. Core entity-relationship model for Report Generation (added for completeness).

Use Case Diagram Snippet - Report Generation

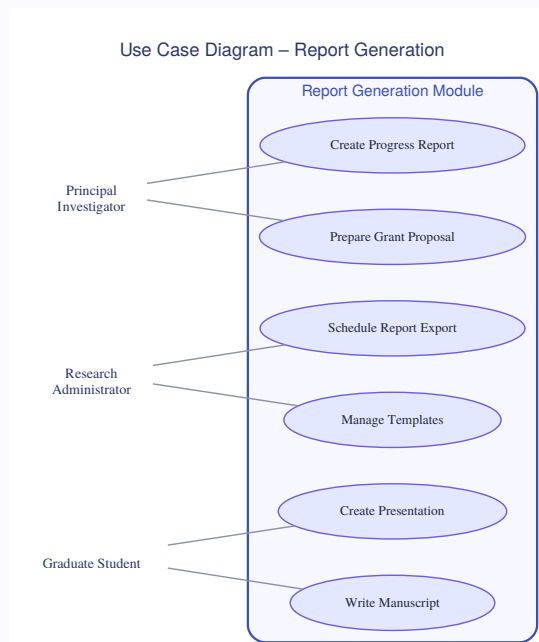


Figure 18. Actors and use cases for the Report Generation module.

Actors: Principal Investigator, Research Administrator, Graduate Student.

Use Cases: Create Progress Report, Write Manuscript, Prepare Grant Proposal, Create Presentation, Schedule Report Export, Manage Templates.

Relationships: Administrator "manages" Template Library; PI "approves" Grant Proposal before submission.

5. Other Non-functional Requirements

5.1 Performance Requirements

- **Response Time:** 95% of page loads and form submissions shall complete within 2 seconds under normal load (50 concurrent users).
- **Throughput:** System shall support sustained throughput of 100 requests per second.
- **Scalability:** Architecture shall support horizontal scaling to accommodate up to 500 concurrent users with additional web nodes and load balancing.
- **Resource Usage:** Memory consumption shall not exceed 500 MB per worker process under peak load.
- **Availability:** System shall achieve 99.5% monthly uptime excluding scheduled maintenance windows.

5.2 Safety Requirements

- **Data Integrity:** System shall use transactions for all multi-step database operations to prevent partial updates.
- **Error Recovery:** System shall automatically recover from transient failures (database connection loss, temporary storage unavailability) without data loss.

- **Input Validation:** All user inputs shall be validated on both client and server sides to prevent injection attacks and data corruption.
- **Safe Defaults:** Default permissions for new projects shall be private (only PI and explicitly invited users can access).
- **Confirmation Actions:** Destructive actions (delete project, delete data) shall require explicit confirmation and optionally require secondary authentication.

5.3 Security Requirements

- **Authentication:** Support for local database, LDAP/Active Directory, and OAuth 2.0 (Google, ORCID) with multi-factor authentication option.
- **Authorization:** Role-Based Access Control (RBAC) with hierarchical roles (Admin > PI > Co-I > RA > Student > Collaborator) and project-specific role overrides.
- **Audit Logging:** All CREATE, UPDATE, DELETE operations on projects, experiments, literature references, and documents shall be logged with user ID, timestamp, IP address, and before/after values (for sensitive fields).
- **Data Protection:**
 - Transmission: TLS 1.2+ enforced
 - Storage: Sensitive fields (SSN, DOB, grant numbers) encrypted at rest using AES-256 with key management service
 - Backups: Encrypted and stored in geographically separate location
- **Vulnerability Management:**
 - Dependencies scanned weekly for known vulnerabilities
 - Annual third-party penetration test
 - Security headers implemented (CSP, HSTS, X-Frame-Options)
- **Data Privacy:**
 - GDPR compliance: right to access, rectify, erase, port personal data
 - HIPAA compliance mode available for health-related research (BAA required)

5.4 Software Quality Attributes

- **Maintainability:**
 - Code adheres to established style guide (PEP 8 for Python, Airbnb for JavaScript)
 - Modular design with clear interfaces between layers
 - Comprehensive API documentation (OpenAPI/Swagger)
 - Technical debt tracked and addressed in each sprint
- **Testability:**
 - Unit test coverage >80% for business logic
 - Integration tests for critical workflows (project creation, experiment logging)
 - Automated UI tests for critical paths (login, data entry, report generation)
 - Test environments mirror production
- **Portability:**

- Containerized via Docker for consistent deployment
- Database abstraction layer allows switching between PostgreSQL/MySQL
- Configuration via environment variables and config files
- **Usability:**
 - Interface follows usability heuristics (visibility, match, control, consistency, error prevention)
 - Accessible per WCAG 2.1 AA (keyboard navigation, ARIA labels, sufficient contrast)
 - Contextual help (tooltips, inline guides) available for all major features
 - User testing conducted with representative researchers each quarter
- **Reliability:**
 - Mean Time Between Failures (MTBF) > 720 hours (30 days)
 - Mean Time To Repair (MTTR) < 2 hours for critical issues
 - Graceful degradation: non-critical features (e.g., real-time collaboration) disabled if supporting services unavailable
- **Interoperability:**
 - RESTful API with JSON/XML responses
 - Webhooks for event notifications (project update, experiment completed)
 - Standard import/export formats (RIS, BibTeX, CSV, JSON, PDF)

6. Other Requirements

6.1 Legal and Compliance

- The system shall comply with all applicable data protection regulations (GDPR, FERPA, HIPAA where applicable).
- The system shall provide mechanisms for obtaining and recording informed consent for human subjects research (if used in that context).
- The system shall not be used for illegal activities or to store illicit data.
- Licensing: The core application shall be open-source (GPLv3 or MIT) to encourage adoption and community contributions. Enterprise features and support may be offered under commercial license.

6.2 Maintenance and Support

- The vendor (or internal team) shall provide:
 - Security patches within 48 hours of critical vulnerability disclosure
 - Bug fixes within 2 weeks for critical bugs, 1 month for non-critical
 - Minor feature updates quarterly
 - Major version releases annually
 - Documentation shall be kept up-to-date with each release.
 - A user forum or support ticket system shall be maintained for community support.
 - Training materials (videos, guides) shall be provided for new releases.

6.3 Acceptance Criteria

- All functional requirements shall be demonstrated in a test environment.
- Performance benchmarks shall be met under specified load conditions.
- Security penetration test shall pass with no critical or high findings.
- User acceptance testing with a panel of 10 researchers shall achieve >80% satisfaction score.
- Documentation shall be complete and accurate.

7. Appendices

Appendix A: Glossary

- **CRUD**: Create, Read, Update, Delete
- **ORM**: Object-Relational Mapping
- **REST**: Representational State Transfer
- **RBAC**: Role-Based Access Control
- **CSL**: Citation Style Language
- **MTBF**: Mean Time Between Failures
- **MTTR**: Mean Time To Repair
- **SLA**: Service Level Agreement
- **OWASP**: Open Web Application Security Project
- **CI/CD**: Continuous Integration/Continuous Deployment

Appendix B: Analysis Models

B.1 DATA FLOW DIAGRAMS (DFD)

See descriptions in each feature section above for context-level and level-1 DFDs.

Key external entities: User, External Services (PubMed, Zotero, etc.), Notification Service (Email/SMS), Storage Systems (File System, Cloud Storage).

Key data stores: User Management, Project Management, Literature Library, Experiment Records, Analysis Results, Document Repository, Audit Log.

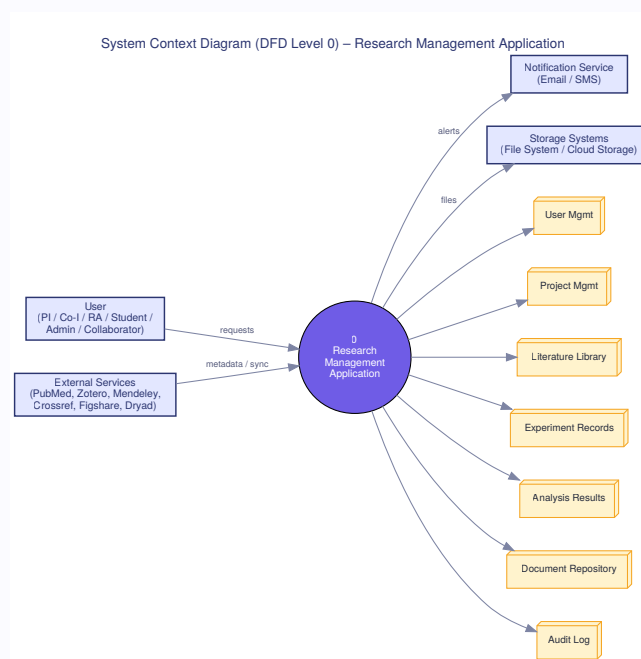


Figure 19. System-wide context DFD (Level 0) for the entire Research Management Application.

B.2 ENTITY-RELATIONSHIP DIAGRAMS (ERD)

See descriptions in each feature section above for entity snippets.

Core entities: User, Project, Milestone, Task, BudgetItem, Reference, ReferenceFile, Annotation, Collection, Protocol, ProtocolStep, ExperimentRun, StepData, Deviation, ExperimentFile, Dataset, DataColumn, Analysis, Visualization, Template, GeneratedDocument, Notification, ActivityLog, Permission, Role.

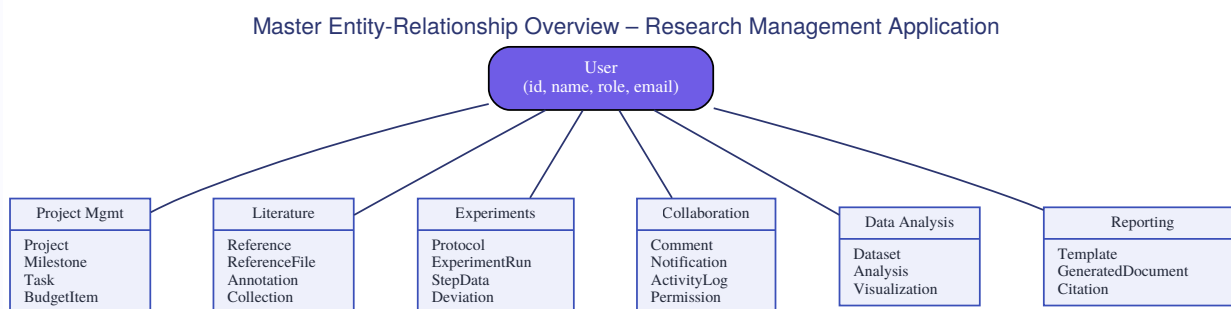


Figure 20. Master entity-relationship overview grouping core entities by module.

B.3 USE CASE DIAGRAMS

See descriptions in each feature section above for use case snippets.

Primary actors: Principal Investigator (PI), Co-Investigator (Co-I), Research Assistant (RA), Student, Administrator, Collaborator.

Use cases grouped by module: Project Management, Literature Management, Experiment Tracking, Collaboration, Data Analysis, Report Generation.

Master Use Case Diagram – Research Management Application

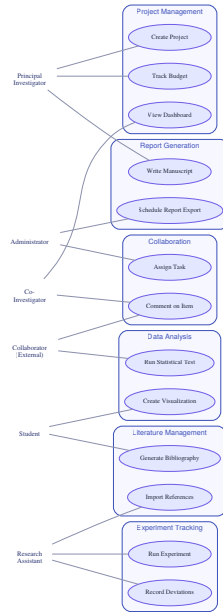


Figure 21. Master use case diagram showing all actors across all modules.

B.4 ACTIVITY DIAGRAMS

Key workflows to model: 1. **New Project Onboarding:** - PI creates project → defines objectives/ milestones → invites team → sets budget → kicks off.

Activity Diagram – New Project Onboarding

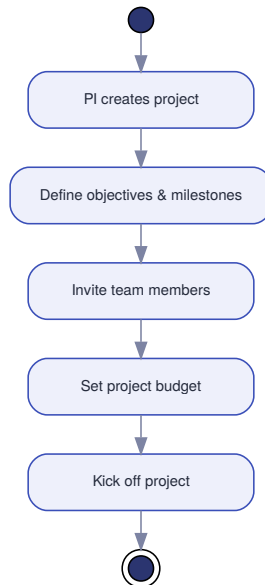


Figure 22. Activity diagram for the New Project Onboarding workflow.

1. **Literature Review Process:** - User imports references → screens for relevance → reads and annotates → organizes into themes → synthesizes findings → cites in manuscript.

Activity Diagram – Literature Review Process

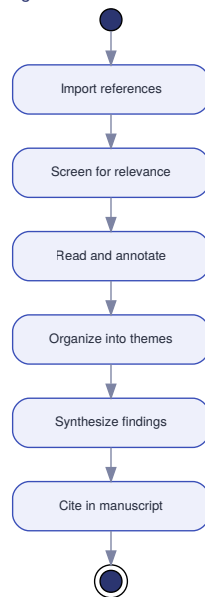


Figure 23. Activity diagram for the Literature Review Process workflow.

1. **Experiment Lifecycle:** - PI approves protocol → RA executes run → logs results and deviations → links data files → PI reviews and approves → data moved to analysis phase.

Activity Diagram – Experiment Lifecycle

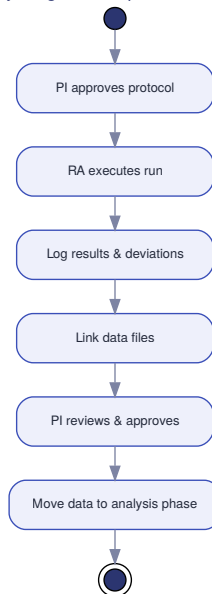


Figure 24. Activity diagram for the Experiment Lifecycle workflow.

1. **Collaborative Writing:** - Author drafts section → inserts dynamic data and citations → shares with co-author for review → incorporates feedback → finalizes → exports to target format.

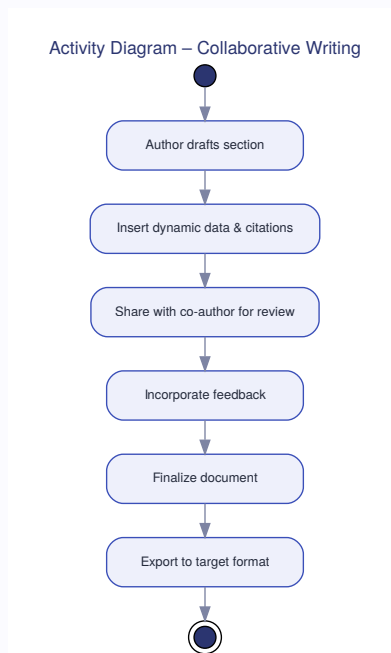


Figure 25. Activity diagram for the Collaborative Writing workflow.

1. **Data Analysis Pipeline:** - Import raw data → clean and transform → exploratory visualization → hypothesis testing → model building → validate results → prepare figures for publication.

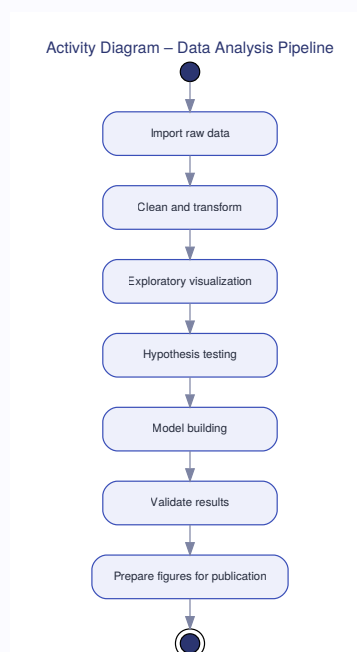


Figure 26. Activity diagram for the Data Analysis Pipeline workflow.

Appendix C: To Be Determined (TBD)

- ☐ Final selection of technology stack (Django vs. Node.js/Express vs. Laravel)
- ☐ Decision on hosting model (institutional servers vs. cloud SaaS)

- ☐ Selection of PDF generation library (WeasyPrint, wkhtmltopdf, LaTeX/Pandoc)
 - ☐ Implementation strategy for real-time collaboration (operational transform vs. CRDTs)
 - ☐ Integration depth with external analysis tools (RStudio Server, JupyterHub)
 - ☐ Mobile application development roadmap (native vs. progressive web app)
 - ☐ Pricing model for potential commercial offering (freemium, tiered, institutional license)
 - ☐ Data retention policies for different research types (clinical vs. basic science)
 - ☐ Disaster recovery plan and recovery time objectives (RTO/RPO)
-

End of Document